

Weekly Report (Week 2)

General Information

- **Group ID:** 55
- **Project Name:** CS202 - Apple Knight
- **GitHub Repository:** <https://github.com/kietanhnguyen2007/CS202-FinalProject>
- **Date range:** 3/7/2026 - 10/7/2026

Detailed Tasks Completed This Week

While our Markdown report provides a high-level summary of the week's progress, this document elaborates on the technical implementations and architectural decisions made by each member.

25125074 – Nguyen Anh Kiet

- **Audio Assets Research & Documentation:** Addressed the missing task from last week by thoroughly researching and compiling `SOUND_ASSET_GUIDE.md` and `DOWNLOAD_SOUND_ASSETS.md`. These documents map all game states, character classes, UI events, and elemental reactions to specific, verified free audio assets (Kenney, OpenGameArt).
- **Dynamic Sound Loading System:** Refactored `GameController::Init()` and implemented `LoadPlayerSounds(CharacterClass)` to dynamically load class-specific audio (Fighter, Knight, MagicCaster, Ninja) into generic internal keys (e.g., `player_attack1`). This successfully eliminated hardcoded class-checking logic inside the combat loops, maintaining the engine's speed and cleanliness.
- **Backend Event Hookups:** Successfully integrated sound triggers into the core engine without altering existing physics or combat logic. Hooked up Player inputs (attacks, dash, ultimate), Enemy/Boss states, Pet actions (summon, dragon fire, ghost heal), and the `ElementalSystem` reactions (Vaporize, Conduct, Overload).

25125037 – Nguyen Trong Tien

- **UI & View Audio Integration:** Connected interface and game state audio triggers (e.g., Level Complete, Game Over) within `ResultView` and `MenuController` to provide immediate and satisfying feedback to the player without coupling the View to the Controller's business logic.
- **Repository Optimization:** Conducted a cleanup pass to remove multiple heavy, unused dummy `.wav` files from the repository. This drastically reduced the project footprint and optimized git clone times for the team.

- **Build System Updates:** Updated `CMakeLists.txt` and `.gitignore` to seamlessly accommodate the new structured audio directories and prevent massive binary assets from clogging the source control in the future.

AI Usage Declaration

- **Debugging & Constraint Enforcement:** AI was used to verify that inserting `PlaySound` calls into the high-frequency fixed-timestep loops (like `UpdateCombat` and `HandlePlayerInput`) did not introduce dynamic allocation or violate the project's strict Zero-Allocation rule.
- **Architectural Blueprints:** AI suggested the strategy of using dynamic search queries for `OpenGameArt` and `Freesound` instead of static links in the Asset Guide to completely eliminate the risk of 404 errors. It also provided the blueprint for mapping class-specific audio to generic keys to keep the controller logic decoupled.

Tasks Planned for Next Week

- Finalize downloading and importing all actual `.wav` and `.ogg` files based on the Asset Guide.
- Thoroughly test the audio system across the TCP network (ensuring Host and Client hear synchronized combat sounds).
- Continue refining boss AI patterns and dual-world level switching mechanics.
- Prepare the final project presentation and demo video.

Issues & Resolutions

- **Issue 1:** Finding high-quality, free, and fitting sound assets for 4 distinct player classes and elemental reactions was highly time-consuming and prone to broken web links.
Resolution: Utilized reliable open-source platforms (`Kenney.nl`) and created a dynamic search link strategy (`DOWNLOAD_SOUND_ASSETS.md`) to guarantee asset availability without 404 errors.
- **Issue 2:** The repository size was ballooning due to large, unused dummy `.wav` files.
Resolution: Purged the unnecessary files and updated `.gitignore` to manage audio assets more effectively outside of direct source control tracking.